EECS 348 Assignment 4

GenAI Analysis Report

Author: Brayden Nelson

Oct 15, 2026

Prompt used for both GenAIs:

The code for this assignment was generated using Gemini 3.1 Pro and Claude Sonnet 5.5 via their respective desktop applications.

Please act as an expert C++ programmer and generate code to solve Sudoku puzzles following these exact specifications:

- Language & Paradigm: Write a recursive object-oriented C++ program that utilizes objects rather than standalone functions.
- Algorithm: Implement a brute-force search utilizing depth-first search with backtracking to exhaustively test candidate values in empty cells.
- Input: The program must read and process five provided text files sequentially (puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt). Blank squares within these files are represented by an underscore character (_).
- Output Details: For every puzzle processed, the console output must display the file name, the original unsolved puzzle, and the final solution.
- Edge Cases: If a puzzle has no valid solution, the program must print "No solution found". If a puzzle contains multiple valid solutions, the program must print all of them.
- Documentation & Comments: Generate a complete prologue comment block at the top of the file that includes the program name, a brief description, inputs, outputs, collaborators (list yourself as the GenAI source), the author's full name (Brayden Nelson), and the creation date. Furthermore, include adequate comments summarizing all major blocks of code, alongside detailed comments on every line explaining its specific function.

Gemini generated code:

```
/*
 * Program Name: Assignment4.cpp
 * Description: An object-oriented recursive C++ program that solves Sudoku puzzles utilizing a depth-first search brute-force algorithm with backtracking.
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 * Outputs: Displays the file name, original unsolved puzzle, and all valid solutions (or "No solution found" if none exist).
 * Collaborators: Gemini (GenAI Source)
 * Author: Brayden Nelson
 * Creation Date: October 15, 2026
 */
```

```cpp
#include <iostream> // Include the standard input/output stream library for console logging
#include <fstream>  // Include the file stream library to handle reading from text files
#include <vector>   // Include the vector container library to store file names
#include <string>   // Include the string library to process text data and file names

using namespace std; // Utilize the standard namespace to omit std:: prefixes

// Class representing the object-oriented Sudoku Solver
class SudokuSolver { // Define the SudokuSolver class
private: // Declare private members for internal state management
    int grid[9][9]; // Declare a 2D integer array to store the 9x9 Sudoku board
    string currentFile; // Declare a string variable to keep track of the current file being processed
    int solutionsFound; // Declare an integer counter to track the number of valid solutions discovered

    // Major Block: Validates if a number can be placed at a specific row and column
    bool isSafe(int row, int col, int num) { // Define the method checking row, column, and subgrid constraints
        for (int x = 0; x < 9; x++) { // Loop 9 times to iterate through every column in the current row
            if (grid[row][x] == num) { // Check if the candidate number already exists in this row
                return false; // Return false immediately because the placement is invalid
            } // End of row conflict check
        } // End of row iteration loop

        for (int x = 0; x < 9; x++) { // Loop 9 times to iterate through every row in the current column
            if (grid[x][col] == num) { // Check if the candidate number
```

```
already exists in this column
                return false; // Return false immediately because the
placement is invalid
            } // End of column conflict check
        } // End of column iteration loop

        int startRow = row - row % 3; // Calculate the top row index of
the 3x3 subgrid containing the cell
        int startCol = col - col % 3; // Calculate the left column index
of the 3x3 subgrid containing the cell

        for (int i = 0; i < 3; i++) { // Loop 3 times to iterate through
the rows of the 3x3 subgrid
            for (int j = 0; j < 3; j++) { // Loop 3 times to iterate
through the columns of the 3x3 subgrid
                if (grid[i + startRow][j + startCol] == num) { // Check
if the candidate number exists in the subgrid
                    return false; // Return false immediately because
the placement is invalid
                } // End of subgrid conflict check
            } // End of subgrid column iteration loop
        } // End of subgrid row iteration loop

        return true; // Return true if no conflicts were found, meaning
the placement is safe
    } // End of isSafe method

    // Major Block: Recursive depth-first search with backtracking to
exhaustively solve the grid
    void solve() { // Define the core recursive backtracking algorithm
        int row = -1; // Initialize the target row index to -1 as a
placeholder
        int col = -1; // Initialize the target column index to -1 as a
placeholder
        bool isEmpty = false; // Initialize a boolean flag to track
whether an empty cell was found

        for (int i = 0; i < 9; i++) { // Loop through all 9 rows of the
grid
```

```
            for (int j = 0; j < 9; j++) { // Loop through all 9 columns
of the grid
                if (grid[i][j] == 0) { // Check if the current cell is
empty (denoted by 0)
                    row = i; // Assign the empty cell's row index to the
target row variable
                    col = j; // Assign the empty cell's column index to
the target col variable
                    isEmpty = true; // Toggle the empty flag to true
since a blank square was located
                    break; // Break out of the inner loop to stop
searching for empty cells
                } // End of empty cell check
            } // End of column iteration loop
            if (isEmpty) { // Check if the empty flag was toggled during
the inner loop
                break; // Break out of the outer loop to begin
processing the identified empty cell
            } // End of empty flag check
        } // End of row iteration loop

        if (!isEmpty) { // Check if no empty cells were found during the
grid traversal
            printGrid(); // Call printGrid to output the fully solved
puzzle
            solutionsFound++; // Increment the solutions counter to
record this successful path
            return; // Return to trigger backtracking in search of
additional valid solutions
        } // End of solved state check

        for (int num = 1; num <= 9; num++) { // Loop through all
possible candidate numbers from 1 to 9
            if (isSafe(row, col, num)) { // Check if the candidate
number can be legally placed in the empty cell
                grid[row][col] = num; // Assign the valid candidate
number to the empty cell
                solve(); // Recursively call the solve method to process
the next empty cell
```

```
                grid[row][col] = 0; // Backtrack by resetting the
current cell to empty (0) to explore other paths
            } // End of placement validity check
        } // End of candidate number loop
    } // End of solve method

    // Major Block: Utility to print the 9x9 grid to the console
    void printGrid() { // Define a helper method to display the puzzle
        for (int i = 0; i < 9; i++) { // Loop through all 9 rows
            for (int j = 0; j < 9; j++) { // Loop through all 9 columns
                if (grid[i][j] == 0) { // Check if the current cell is
considered empty
                    cout << "_ "; // Output an underscore and a space to
visually represent a blank square
                } else { // Handle the case where the cell contains a
number
                    cout << grid[i][j] << " "; // Output the stored
number followed by a space
                } // End of cell value check
            } // End of column output loop
            cout << endl; // Output a newline character at the end of
each printed row
        } // End of row output loop
        cout << endl; // Output an additional newline for spacing below
the grid
    } // End of printGrid method

public: // Declare public members accessible outside the class
    // Major Block: Constructor to initialize class variables
    SudokuSolver(string filename) { // Define the parameterized
constructor accepting a file name
        currentFile = filename; // Store the provided file name in the
class's state
        solutionsFound = 0; // Initialize the total solutions counter to
zero
    } // End of constructor

    // Major Block: Main driver to read files, establish the board, and
execute the solver
```

```
    void processPuzzle() { // Define the public method handling the
end-to-end processing of a puzzle
        ifstream file(currentFile); // Initialize an input file stream
with the designated file name
        if (!file.is_open()) { // Verify if the file stream failed to
open the document
            cout << "Error opening file: " << currentFile << endl; //
Print an error message if the file is inaccessible
            return; // Exit the method prematurely due to the file error
        } // End of file open check

        string val; // Declare a string variable to temporarily hold
each parsed token from the file
        for (int i = 0; i < 9; i++) { // Loop through the 9 expected
rows of input
            for (int j = 0; j < 9; j++) { // Loop through the 9 expected
columns of input
                file >> val; // Extract the next string token from the
file stream
                if (val == "_") { // Check if the extracted token is an
underscore representing a blank
                    grid[i][j] = 0; // Populate the internal grid cell
with 0 to mark it as empty
                } else { // Handle the case where the extracted token is
a numerical digit
                    grid[i][j] = stoi(val); // Convert the string token
to an integer and store it in the grid
                } // End of token evaluation
            } // End of column parsing loop
        } // End of row parsing loop
        file.close(); // Close the input file stream to free system
resources

        cout << "File: " << currentFile << endl; // Output the name of
the file currently being processed
        cout << "Original Puzzle:" << endl; // Output a header for the
initial puzzle state
        printGrid(); // Call the printGrid method to display the parsed
unsolved board
```

```
        cout << "Solution(s):" << endl; // Output a header before
generating the solutions
        solve(); // Trigger the recursive depth-first search algorithm

        if (solutionsFound == 0) { // Check if the backtracking
algorithm failed to find any valid complete boards
            cout << "No solution found" << endl << endl; // Print the
required fallback message per requirements
        } // End of no solution check
    } // End of processPuzzle method
}; // End of SudokuSolver class

// Major Block: Program entry point
int main() { // Define the main execution function of the C++ program
    vector<string> files = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt",
"puzzle4.txt", "puzzle5.txt"}; // Initialize a vector containing all
required file names

    for (const string& file : files) { // Iterate sequentially through
each file name string in the vector
        SudokuSolver solver(file); // Instantiate a new SudokuSolver
object targeting the current file
        solver.processPuzzle(); // Invoke the processing sequence to
read, print, and solve the puzzle
        cout << "-----------------------------------" << endl; // Output
a visual separator between different file results
    } // End of file iteration loop

    return 0; // Return 0 to the operating system, signifying successful
execution
} // End of main function
```

Claude generated code:

```
//
================================================================================
// Program Name: EECS 348 Assignment 4 - Recursive Object-Oriented
Sudoku Solver
// Description : Reads five Sudoku puzzle files, then uses a brute-force
//               depth-first search with backtracking to find EVERY
valid
//               solution for each puzzle. Prints the file name, the
original
//               puzzle, and every solution (or "No solution found").
// Inputs      : puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt,
puzzle5.txt
//               (9 rows of 9 space-separated entries; digits 1-9, '_' =
blank)
// Outputs     : Console text showing, for each puzzle, the file name,
the
//               unsolved grid, and all solutions found (or a
no-solution note)
// Collaborators: Generated by a GenAI assistant (Claude, by Anthropic);
the
//               prompt was written by the author.
// Author      : Brayden Nelson
// Creation Date: October 15, 2026
//
================================================================================

#include <iostream>   // for console output (std::cout)
#include <fstream>    // for reading puzzle files (std::ifstream)
#include <string>     // for std::string file names and tokens
#include <vector>     // for std::vector to store all solutions
#include <array>      // for std::array to hold a fixed 9x9 grid

// Alias for a 9x9 grid of ints so the type is short and readable.
using Grid = std::array<std::array<int, 9>, 9>;
```

```
//
-------------------------------------------------------------------------
---
// Class SudokuPuzzle: owns one puzzle's grid, solves it recursively,
and
// stores/prints every solution it discovers.
//
-------------------------------------------------------------------------
---
class SudokuPuzzle {
private:
    Grid grid_;                    // current working grid (0 means
blank)
    Grid original_;                // untouched copy of the puzzle as
loaded
    std::vector<Grid> solutions_;  // every completed grid found by the
search
    std::string fileName_;         // name of the file the puzzle came
from
    bool loaded_;                  // true only if the file was read
correctly

    // ---- Constraint check
-----------------------------------------------
    // Returns true if placing 'value' at (row, col) breaks no row,
column,
    // or 3x3 block rule given the current contents of grid_.
    bool isSafe(int row, int col, int value) const {
        for (int i = 0; i < 9; ++i) {                       // scan 9
positions
            if (grid_[row][i] == value) return false;       // value
already in row
            if (grid_[i][col] == value) return false;       // value
already in column
        }
        int blockRow = (row / 3) * 3;                       // top row
of this block
        int blockCol = (col / 3) * 3;                       // left
column of this block
```

```
        for (int r = 0; r < 3; ++r) {                          // walk rows
of block
            for (int c = 0; c < 3; ++c) {                      // walk
columns of block
                if (grid_[blockRow + r][blockCol + c] == value) return
false; // in block
            }
        }
        return true;                                           // no
conflict found
    }

    // ---- Initial-clue check
-----------------------------------------------
    // Verifies the starting clues don't already contradict each other,
    // so an invalid puzzle is reported as having no solution.
    bool cluesAreConsistent() {
        for (int r = 0; r < 9; ++r) {                          // every row
            for (int c = 0; c < 9; ++c) {                      // every
column
                int v = grid_[r][c];                           // clue at
this cell
                if (v == 0) continue;                          // skip
blanks
                grid_[r][c] = 0;                               //
temporarily remove it
                bool ok = isSafe(r, c, v);                     // would it
still be legal?
                grid_[r][c] = v;                               // put it
back
                if (!ok) return false;                         // duplicate
clue found
            }
        }
        return true;                                           // all clues
agree
    }

    // ---- Recursive depth-first search with backtracking
```

```
-------------------
    // 'pos' is a cell index 0..80 (row = pos/9, col = pos%9). Each call
    // handles the first blank cell at or after 'pos'.
    void search(int pos) {
        while (pos < 81 && grid_[pos / 9][pos % 9] != 0) ++pos; // skip
filled cells
        if (pos == 81) {                                        // every
cell is filled
            solutions_.push_back(grid_);                        // record
this solution
            return;                                             // backtrack
to find more
        }
        int row = pos / 9;                                      // row of
blank cell
        int col = pos % 9;                                      // column of
blank cell
        for (int value = 1; value <= 9; ++value) {              // try each
candidate
            if (isSafe(row, col, value)) {                      // only
legal candidates
                grid_[row][col] = value;                        // place the
candidate
                search(pos + 1);                                // recurse
to next cell
                grid_[row][col] = 0;                            // undo
(backtrack)
            }
        }
        // If no candidate worked, the loop ends and we return, which
tells the
        // caller its earlier choice was wrong.
    }

    // ---- Grid printer
-----------------------------------------------------
    // Prints any grid, showing '_' for blanks, one row per line.
    static void printGrid(const Grid& g) {
        for (int r = 0; r < 9; ++r) {                           // each row
```

```
            for (int c = 0; c < 9; ++c) {                    // each
column
                if (g[r][c] == 0) std::cout << "_ ";         // blank
cell
                else std::cout << g[r][c] << " ";            // filled
cell
            }
            std::cout << "\n";                               // end the
row
        }
    }

public:
    // Constructor: starts with an empty, not-yet-loaded puzzle.
    SudokuPuzzle() : grid_{}, original_{}, loaded_(false) {}

    // Reads the puzzle from a text file; returns false on any problem.
    bool load(const std::string& fileName) {
        fileName_ = fileName;                                // remember
the name
        solutions_.clear();                                  // discard
old results
        std::ifstream in(fileName);                          // open the
file
        if (!in) return false;                               // could not
open it
        for (int r = 0; r < 9; ++r) {                        // read 9
rows
            for (int c = 0; c < 9; ++c) {                    // read 9
cells per row
                std::string token;                           // one entry
from file
                if (!(in >> token)) return false;            // file
ended too early
                if (token == "_") grid_[r][c] = 0;           //
underscore = blank
                else if (token.size() == 1 && token[0] >= '1' &&
token[0] <= '9')
                    grid_[r][c] = token[0] - '0';            // convert
```

```
char to int
                else return false;                                  //
unexpected token
            }
        }
        original_ = grid_;                                          // keep a
copy to print
        loaded_ = true;                                             // mark
success
        return true;                                                // done
    }

    // Starts the recursive search and stores all solutions.
    void solve() {
        solutions_.clear();                                         // reset any
old results
        if (!loaded_) return;                                       // nothing
to solve
        if (cluesAreConsistent()) search(0);                        // search
only if sane
    }

    // Prints file name, original puzzle, and all solutions (or none
found).
    void report() const {
        std::cout << "File: " << fileName_ << "\n";                 // file name
        if (!loaded_) {                                             // read
failure
            std::cout << "Could not read a valid puzzle from this
file.\n\n";
            return;                                                 // nothing
more to show
        }
        std::cout << "Puzzle:\n";                                   // heading
        printGrid(original_);                                       // show
unsolved grid
        if (solutions_.empty()) {                                   // search
found nothing
            std::cout << "No solution found\n\n";                   // required
```

```
message
            return;                                          // finished
        }
        for (std::size_t i = 0; i < solutions_.size(); ++i) { // each
solution
            std::cout << "Solution " << (i + 1) << ":\n";    // number it
            printGrid(solutions_[i]);                        // print the
grid
        }
        std::cout << "Solutions found: " << solutions_.size() << "\n\n";
// total
    }
};

//
---------------------------------------------------------------------------
---
// main: creates one SudokuPuzzle object per file and runs them in
order.
//
---------------------------------------------------------------------------
---
int main() {
    const std::string files[5] = {                           // the five
input files
        "puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt",
"puzzle5.txt"
    };
    for (const std::string& name : files) {                  // process
sequentially
        SudokuPuzzle puzzle;                                 // new
puzzle object
        puzzle.load(name);                                   // read the
file
        puzzle.solve();                                      // run the
recursive search
        puzzle.report();                                     // print the
results
    }
```

```
    return 0;                                                    // normal
exit
}
```

Analysis:

Correctness -

- **Claude:** The program effectively handles depth-first search backtracking. It also verifies that the initial clues provided in the text file are consistent, using the cluesAreConsistent() function, before searching. Checking first improves the codes robustness.
- **Gemini:** The program follows the same depth-first search backtracking approach as the Claude program, but does not validate the initial board consistency before executing the solve() function. This could potentially waste cycles or proceed to try and fun on a dataset that is flawed.

Execution Time -

- **Claude:** The program runs with a time complexity of O(9^m), where m is the number of empty cells in the puzzle. This is the worst-case time complexity since we are using the brute force method. This is optimized by the code by using a linear pointer to reduce the search time for the next blank cell to an O(1) average operation per recursive call.
- **Gemini:** The program has the same O(9^m) structural time complexity as the Claude generated program, but implements an O(9^2) iteration every time the solve() function is called to find an empty cell. This is a redundant nested loop and significantly increases the constant time execution constraints.

Space Complexity -

- **Claude:** The base space complexity for the recursive call stack is O(m), where m is the maximum recursion depth (the number of empty cells). But by recording every finished grid state directly into a dynamic vector, its overall space complexity scales to O(S), where S is the total number of valid solutions.
- **Gemini:** The program operates with optimal the optimal space complexity of O(m) for the recursive call stack. It prints valid solutions directly to the console using the printGrid() function as they are discovered, preventing the memory footprint from scaling to O(S), like the Claude code does, because it does not store the found solutions.

Maintainability -

- **Claude:** The code takes advantage of C++ standard library structure by using std::array instead of C-style arrays. Functions are correctly broken apart into modular steps [load(), solve(), and report() functions], which greatly increases the readability and maintainability of the code. It also includes detailed comments explaining what each line of code is doing.

- **Gemini:** The code relies heavily on raw array indices and processes the entire file structure inside the processPuzzle() function while intermixing solver() initializations and UI prints. The lack of separation hurts the maintainability of this code.

Code Selection:

I selected the Claude code as the base for this assignment. It is more robust since it uses std::array which prevents boundary decay and separates the problem logic into dedicated modular methods [load(), solve(), and report() functions]. This code also validates the puzzles internal consistency before running the search by using the cluesAreConsistent() function.

Claude Code Improvement:

- **Correctness**: I improved the file handling inside the load() function to reject invalid tokens and manage stream failures better when testing input files that are not formatted correctly.
- **Execution Time**: I kept the mathematical sub grid formula [int blockRow = (row / 3) * 3], since it validates the sub grid safety in O(1) time, which avoids redundant modulus calculations and O(N^2) time nested loops.
- **Space Complexity**: I improved the baseline space complexity by making sure the printGrid() function passes the 9x9 matrix by constant reference. This avoids the O(N^2) memory duplication step on every print execution, maintaining the O(m) space efficiency of the call stack.
- **Maintainability**: I adjusted the prologue comments to match the assignment rubric. I also changed variable names to standardize the internal structures to camelCase.